

ZDPatch: Execution-Guided Program Repair from Student Submission Trajectories

ANONYMOUS AUTHOR(S)

Automated program repair for programming assignments typically treats a student’s latest submission as an isolated buggy program. This snapshot omits the revisions, repeated failures, and partial improvements that led to the current code. We present ZDPATCH, a repair framework that converts authentic submission trajectories into an execution-ordered portfolio of three repair policies. Verdict and testcase improvements define finite execution-evidence orders; validation execution then selects three checkpoints by exact submodular coverage maximization, while the online controller guarantees observed-test non-regression and optional AST edit-budget compliance. The PROGRESS adapter learns testcase-level Pareto improvements, STRICT learns coarse-verdict improvements, and ANSWER maps failed submissions to the terminal accepted program. We evaluate Qwen2.5-Coder-7B-based adapters on 997 held-out trajectories from 328 test problems drawn from a 492-problem Seen pool, plus 250 trajectories from 54 problem-held-out problems. On Seen problems, the selected portfolio improves repair rate from 30.7% for trajectory-prompted zero-shot to 61.5%; a same-target three-seed Answer control reaches 61.7%, identifying independent checkpoint diversity rather than target heterogeneity as the supported mechanism. On Unseen problems, selection reaches 74.8% RR versus 62.0% for zero-shot and 73.2% for Answer-3Seed. The one-per-relation constraint reduces Seen coverage, whereas budget-indexed unconstrained selection improves the mean Unseen edit-budget frontier by 2.60 points [0.38, 5.03] over Answer-3Seed. A target-matched ablation finds no stable benefit from serializing earlier submissions, and all primary differences use problem-clustered inference. Against always-three LSGen under the same edit gate, its mean Seen budget frontier is +14.68 points [12.03, 17.43]. On student-held-out CodeWorkout Java, unconstrained selection transfers at 84.0% RR but is statistically indistinguishable from Answer-3Seed at 84.5%, showing external repair transfer without a relation-specific coverage effect.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Applied computing** → *Education*; • **Computing methodologies** → *Natural language generation*.

Additional Key Words and Phrases: automated program repair, programming education, large language models, submission trajectories, execution-guided repair

ACM Reference Format:

Anonymous Author(s). 2027. ZDPatch: Execution-Guided Program Repair from Student Submission Trajectories. In *Proceedings of ACM International Conference on the Foundations of Software Engineering (FSE '27)*. ACM, New York, NY, USA, 21 pages.

1 Introduction

Programming students iteratively write, submit, execute, and revise code. Online judges make this loop scalable, but their feedback is usually limited to coarse verdicts such as Wrong Answer, Runtime Error, or Time Limit Exceeded. A verdict reports failure without explaining whether the latest revision made progress, repeated an earlier defect, or moved away from a viable solution. Instructors cannot inspect every trajectory and author an individualized repair at the scale of modern programming courses.

Automated program repair (APR) offers executable, code-specific feedback. The field spans search-based generate-and-validate repair, learned transformations, and test-guided validation [Gazzola et al. 2019; Le Goues et al. 2012; Monperrus 2018; Weimer et al. 2009]. Systems for programming assignments cluster peer solutions, align a student’s code to a reference, or synthesize a fixed program [Gulwani et al. 2018; Hu et al. 2020; Wang et al. 2018]. More recent systems use language models, execution diagnostics, retrieved examples, and iterative conversations [Joshi et al. 2023; Liu et al. 2024; Orvalho et al. 2025; Yang et al. 2024; Zhang et al. 2024]. Most of these approaches

condition on the latest buggy program, possibly augmented with tests or peer programs. The earlier submissions produced by the same student are rarely treated as first-class repair context.

The motivation for using history is straightforward but unverified. Two students may arrive at similar current programs through different revisions. Their histories expose repeated errors, attempted edits, and states that they demonstrably reached. In educational terms, such reachable improvements resemble the Zone of Proximal Development (ZPD): states attainable with appropriate support but not yet reached independently [Chounta et al. 2017; Cole et al. 1978; Murray and Arroyo 2002]. We therefore ask whether repair can exploit a student’s observed trajectory to produce a useful next program rather than immediately replacing it with an arbitrary correct solution. We use ZPD as a design motivation, not as a measured psychological construct or a claim about learning outcomes.

We introduce ZPDPATCH, illustrated in Figure 1. It automatically converts authentic submission trajectories into three supervised repair datasets. PROGRESS retains verdict improvements and same-verdict transitions whose per-test outcomes improve monotonically. STRICT retains only transitions that improve the online-judge verdict. ANSWER pairs every failed submission with the trajectory’s final accepted program. Three seeds per target relation produce nine independently trained QLoRA checkpoints over the same code language model and prompt. Validation execution selects three complementary checkpoints, which are invoked from narrower to broader supervision. ZPDPATCH executes each candidate, stops on an eligible acceptance, and otherwise selects a non-regressive candidate using test pass rate and AST tree edit distance (TED).

Our evaluation separates three mechanisms that are easy to conflate: learned relational targets, heterogeneous policy composition, and the serialization of prior submissions. The complete system substantially improves over zero-shot on both seen and problem-held-out data, and sequential escalation outperforms each adapter alone. Target-matched retraining finds no stable causal benefit from serializing earlier submissions; both conditions still use targets mined from trajectories, so this null result neither establishes nor refutes relational supervision as the source of the complete-system gain. We therefore test policy heterogeneity directly against three independently trained Answer adapters that share examples, targets, and decoding but differ in training seed.

This paper makes the following contributions:

- We formulate trajectory-mined repair as finite execution-evidence orders plus terminal closure, and audit every materialized supervision transition against those definitions.
- We pose size-three checkpoint choice as exact submodular validation-coverage maximization, derive a finite-family problem-level selection guarantee, and design an execution controller with observed-test non-regression and structural edit-budget compliance by construction.
- We conduct a problem-familiarity-aware evaluation against zero-shot and retrieval-based repair, including the missing same-target three-seed control, patch-budget curves, target-matched history ablations, problem-clustered analyses, and student-held-out Java transfer.

2 Background and Related Work

2.1 Structure- and Reference-Based Educational Repair

Educational APR can exploit many programs written for one specification. CLARA clusters and aligns programs before extracting a repair; SARFGEN searches and aligns reference programs; Refactory normalizes refactorings before comparison [Gulwani et al. 2018; Hu et al. 2020; Wang et al. 2018]. Embeddings, solution-space models, and visualization propagate annotations or expose recurring strategies, while multi-function and diversified methods broaden the target beyond one correction [Choi et al. 2021; Choi and Lee 2025; Glassman et al. 2015; Heo et al. 2023; Piech et al.

2015b; Rivers and Koedinger 2017; Yi et al. 2017]. These methods provide strong assignment-specific evidence and establish that execution correctness and relationship to the student’s program are separate repair properties.

LSGen is our reference-rich baseline. It filters historical repair pairs, retrieves by edit information, generates, and may retrieve again from the new state [Dai et al. 2026]. We retain its access to other users’ Seen-Train solutions for the target problem. That information is unavailable on genuinely new assignments, so LSGen is not evaluated on problem-held-out Unseen tasks. This separates repository-mature repair from repair using only a learned model, statement, and tests.

2.2 Neural and Large-Language-Model Repair

Neural repair learned transformations from repository histories and context-aware translation before pretrained code models enabled zero-shot and fine-tuned repair [Jiang et al. 2023; Lutellier et al. 2020; Tufano et al. 2019; Xia and Zhang 2022; Yasunaga and Liang 2021]. RING, PyDex, and FastFixer cast educational repair as conditional generation [Joshi et al. 2023; Liu et al. 2024; Zhang et al. 2022, 2024]; retrieval and localization further constrain generation with peer programs or high-precision syntax feedback [Phung et al. 2023; Zhao et al. 2024]. These systems cover programs that do not align cleanly with a reference, but make correctness and patch scope external validation concerns.

Execution-aware methods address those concerns with signals independent of model confidence. SelfAPR learns from test diagnostics; FixEval makes execution the repair-evaluation criterion; CREF carries diagnostics through a repair conversation; counterexample-guided repair combines generation, fault localization, and MaxSAT [Haque et al. 2023; Orvalho et al. 2025; Yang et al. 2024; Ye et al. 2022]. ZPDPATCH likewise executes candidates, but introduces a different unit of supervision and control. Authentic trajectories define nested improvement relations rather than synthetic corruptions or manually assigned repair roles. Independently trained members share a base model, generated candidates remain conditionally independent given authentic observations, and an explicit selector includes the unchanged program. Thus a failed generation cannot force a regression or contaminate another policy’s input distribution.

2.3 Adaptive Feedback, ZPD, and Submission Histories

Programming feedback ranges from correctness reports to next-step hints and complete repairs; feedback research emphasizes timeliness, task focus, and actionable information rather than correctness alone [Hattie and Timperley 2007; Keuning et al. 2018; Narciss 2008; Shute 2008]. Tutoring research studies help seeking and assistance at a productive granularity; iSnap and LLM hint systems similarly expose next-step or multi-level support [Aleven and Koedinger 2000; Heickal and Lan 2024; Price et al. 2017; Roest et al. 2024; Xiao et al. 2024]. The ZPD motivates support between independent and assisted performance [Chounta et al. 2017; Cole et al. 1978; Murray and Arroyo 2002; Schulze Bernd and Chounta 2024]. We operationalize only observed reachability: a later improving submission proves that this user reached that program state, not that showing the edit causes learning or is optimal scaffolding.

Submission sequences also support knowledge tracing, which estimates latent mastery from interaction histories [Corbett and Anderson 1994; Ghosh et al. 2020; Piech et al. 2015a]. ZPDPATCH instead orders source-code states using execution evidence; it neither infers mastery nor predicts a learner response. This distinction permits fully automatic repair targets while delimiting the contribution to program behavior rather than educational outcomes.

2.4 Positioning of This Work

Table 1 locates the contribution along evidence and composition axes. Unlike reference alignment and LSGen, ZPDPATCH does not require a same-problem solution repository; unlike prior LLM and iterative repair, its targets are mined as ordered transitions from the same user’s authentic submissions. Its technical contribution is the combination of this execution-evidence order, policy-specific parameterization, and certified sequential selection.

3 Problem Formulation

For problem p , let a student’s i -th submission be $s_i = (c_i, v_i)$, where c_i is the complete source code and v_i is the online-judge verdict. A submission trajectory $\tau = [s_1, s_2, \dots, s_n]$ is chronologically ordered and terminates at the first accepted submission. Let d_p contain the problem statement and resource constraints, and let $\mathcal{E}_p$ be the available test suite. Re-executing c_i gives a per-test outcome vector $o_i = (o_i(e))_{e \in \mathcal{E}_p}$.

A repair example consists of problem context d_p , an observed sequence h , and a complete target program y . Adapter-specific rules decide which submissions remain in h and which later submission supplies y ; the two need not be adjacent in the original trajectory. Given $x = (d_p, h)$, a model generates one complete program $\hat{c}$.

We call a later state *reachable* when it occurs in the same student’s observed trajectory and satisfies an adapter-specific improvement rule. Reachability is an empirical property of the data, not a guarantee that the transition is minimal, comprehensible, or caused by instructional support. Our design hypothesis is that learning from such transitions can produce repairs that preserve more of the current solution than direct answer generation.

4 Approach

4.1 Design Requirements

Four requirements shape the method: *automatic supervision* recoverable from submissions and tests; *target diversity* spanning intermediate improvement and accepted solutions; *executable validation* independent of model confidence; and *non-regression* when no candidate improves observed behavior. We separate these offline and online decisions. Label relations determine what each adapter learns; execution and selection determine what the system may return. A noisy target can influence generation without automatically overriding a better current program.

4.2 Trajectory Construction

We group submissions by problem and student, sort by time, and stop at first acceptance. After indentation/comment/blank-line normalization, we remove only consecutive duplicates; non-consecutive revisits remain evidence of distinct attempts. Complete source, verdict, and order are preserved, with no maximum history or transition count. Every state is re-executed. Cache keys bind problem, normalized code, test-suite identity, timeout, and memory limit, preventing incompatible reuse. Runner failures receive an explicit most-severe outcome, and Progress construction requires a complete cache.

4.3 Adapter-Specific Supervision

We order verdict severity as $\text{sev}(\text{AC}) = 0$, $\text{sev}(\text{WA}) = 1$, $\text{sev}(\text{TLE}/\text{MLE}) = 2$, $\text{sev}(\text{RE}) = 3$, $\text{sev}(\text{CE}) = 4$, and 5 for internal failures. For equal, non-empty test keys, $o_t \preceq_{\text{sev}} o_r$ iff no test outcome in o_t is more severe than its counterpart in o_r . Test-case progress is the Pareto condition

$$\text{TCProg}(r, t) = \mathbf{1}[o_t \preceq_{\text{sev}} o_r \wedge o_t \neq o_r]. \quad (1)$$

Table 1. High-level positioning of educational repair approaches. “Same-problem refs.” denotes a database of solutions or repair pairs for the target assignment; “trajectory supervision” denotes training targets mined from ordered submissions by the same user.

Approach family	Refs.	Exec.	Traj.	Stages
Structural alignment [Gulwani et al. 2018; Hu et al. 2020; Wang et al. 2018]	Yes	Opt.	No	No
LLM generation [Joshi et al. 2023; Liu et al. 2024; Zhang et al. 2024]	Opt.	Opt.	No	No
Iterative repair [Orvalho et al. 2025; Yang et al. 2024]	Opt.	Yes	No	Yes
LSGen [Dai et al. 2026]	Yes	Yes	No	Yes
ZPDPATCH	No	Yes	Yes	Yes

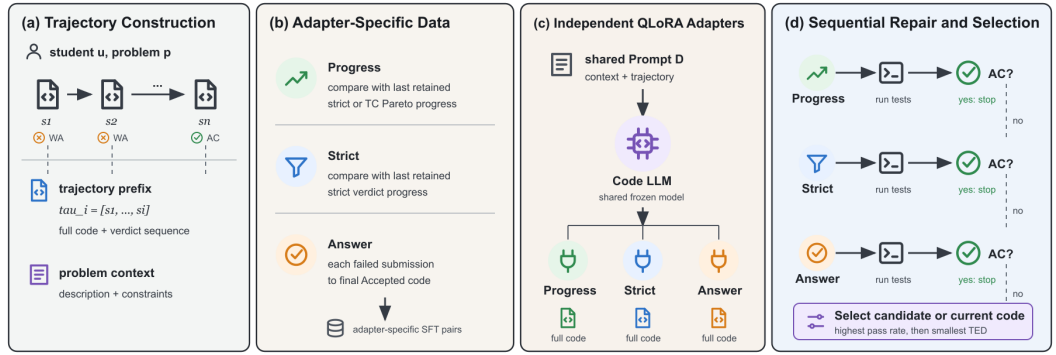


Fig. 1. Overview of trajectory supervision and execution control. The displayed P–S–A route is the one-per-relation control; the deployed system trains three seeds per relation and uses validation execution to select any three checkpoints, so a relation may repeat. Execution outcomes determine early stopping and final selection.

Answer. If s_n is accepted, every preceding non-accepted submission is independently paired with c_n . For $\tau = [s_1, \dots, s_9]$, the rule produces $([s_1], c_9), \dots, ([s_8], c_9)$. Thus, ANSWER learns a direct failed-program-to-answer mapping and receives no multi-submission history.

Strict. Starting with $R = [s_1]$, we scan the trajectory and append s_t only when its verdict is strictly better than the last retained submission $s_r = R[-1]$: $\text{sev}(v_t) < \text{sev}(v_r)$. If $R = [r_1, \dots, r_m]$, we create $([r_1, \dots, r_{k-1}], c_{r_k})$ for every $k = 2, \dots, m$. The comparison is against the last retained state, not necessarily the immediately preceding original submission.

Progress. PROGRESS uses the same retained scan but appends s_t when either its verdict strictly improves or $v_t = v_r$ and $\text{TCProg}(r, t) = 1$. It therefore captures monotonic per-test progress that is hidden by an unchanged coarse verdict. Prefix-target construction is otherwise identical to STRICT.

The datasets are constructed independently and may overlap. A strict transition is also a progress transition, and final accepted transitions can occur in both. The objectives nevertheless differ: ANSWER learns direct completion from one failed program, whereas STRICT and PROGRESS learn transitions from retained prefixes.

The common retained-prefix procedure scans $s_2, \dots, s_n$, compares each state with $R[-1]$, and, whenever the corresponding predicate above holds, emits (R, c_t) before appending s_t to R . Comparing with the last retained rather than immediately previous raw state makes R monotonic under the

selected relation. A trajectory with m retained states yields all $m - 1$ prefixes, so early transitions are not discarded.

4.4 Execution-Evidence Order

The three rules are not unrelated task labels. They form two nested execution-evidence orders plus a terminal closure. Let the evidence of submission (s) be $(e(s)=v(s),o(s))$ over the fixed testcase universe $\mathcal{E}_p$. Define the strict relation

$$s \prec_S t \iff \text{sev}(v(t)) < \text{sev}(v(s)), \quad (2)$$

and the progress relation

$$s \prec_P t \iff s \prec_S t \vee (v(s) = v(t) \wedge o(t) \prec_\times o(s)), \quad (3)$$

where $o(t) \prec_\times o(s)$ is the strict product order: every testcase outcome is no worse and at least one is better under severity. Strict and Progress retain a chain under $\prec_S$ and $\prec_P$, respectively. Answer is the terminal closure $A(s) = s_n$ from each failed state to the first accepted state; it is intentionally not an intermediate order.

This view makes the target granularity explicit. Strict observes only a projection of execution evidence onto the coarse verdict. Progress refines that projection with the testcase product order. Answer discards intermediate ordering and connects a failed state to the absorbing accepted state. Adapter specialization therefore corresponds to distinct relations over the same trajectory rather than to role words inserted in prompts.

PROPOSITION 1 (FINITE MONOTONE SUPERVISION). *For a problem with T tests, every transition retained by Progress strictly decreases the lexicographic rank $\rho(s) = (\text{sev}(v(s)), \sum_{e \in \mathcal{E}_p} \text{sev}(o(s, e)))$. Consequently, retained Progress chains contain no cycles and have length at most $6(5T + 1)$. Strict chains satisfy the same result through their first rank component.*

PROOF. A Strict transition decreases the first component, independently of the second. A same-verdict Progress transition is Pareto non-worsening and strict on at least one testcase, so it preserves the first component and decreases the sum. Thus every retained transition strictly decreases ρ . There are six verdict-severity values and $5T + 1$ possible sums, which gives the finite bound. $\square$

Because $\prec_S \subseteq \prec_P$, every Strict target event is also a Progress event. Moreover, for any failed state, the first Progress-improving state cannot occur after the first Strict-improving state, and acceptance closes both relations. These inclusion facts determine the fine-to-broad policy order; they are construction invariants rather than claims about learned behavior.

Order versus scalar reward. A passed-test count can improve while breaking a previously passing testcase; Equation 1 refuses this compensation. Coarse verdict is the first coordinate and non-compensatory testcase severity refines equal verdicts, so the rank proves termination while the product order records which observations justify a transition. Execution certifies retained supervision, not model behavior on new inputs; the online controller must still test every candidate and retain the original fallback. The order is defined only over one fixed testcase universe and severity map, so cache keys bind tests and resource limits and Progress fails closed on incomplete vectors. Otherwise a missing test could turn a regression into apparent Pareto progress.

4.5 Prompt and Adapter Training

All adapters share one role-neutral prompt: statement and limits, chronologically retained complete submissions with verdict/outcome/edit summaries, and the current submission requesting one complete solution in the dataset's language and format. No instruction names a policy or asks for a

particular edit size, so specialization can arise only from supervision. Only assistant target tokens contribute to loss.

For adapter $a \in \{\text{prog, strict, ans}\}$, let $\mathcal{D}_a$ be its dataset, x_i the prompt, and y_i the target tokens. We minimize the target-only supervised fine-tuning objective

$$\mathcal{L}_a = -\frac{1}{|\mathcal{D}_a|} \sum_{(x_i, y_i) \in \mathcal{D}_a} \frac{1}{|y_i|} \sum_{k=1}^{|y_i|} \log P_{\theta, \phi_a}(y_{i,k} \mid y_{i,<k}, x_i), \quad (4)$$

where θ is the frozen base model and ϕ_a is an adapter's QLoRA parameter set. Prompt tokens are masked from the loss. The adapters are trained and stored independently.

4.6 Validation-Selected Execution Portfolio

Training loss ranks checkpoints within one target distribution, but it does not measure the cross-policy repair complementarity used at deployment. We therefore select the portfolio using execution on validation problems. Let $V = V_P \dot{\cup} V_S \dot{\cup} V_A$ be independently trained Progress, Strict, and Answer checkpoints and let $R_v \subseteq U$ be the validation instances repaired by checkpoint v . For a portfolio S , define execution coverage

$$f(S) = |\bigcup_{v \in S} R_v|/|U|. \quad (5)$$

The deployed search family is $\mathcal{F}_3 = \{S \subseteq V : |S| = 3\}$. It allows validation evidence to choose both target relations and training seeds rather than assuming that one checkpoint per relation is optimal. We retain $\mathcal{F}_{\text{rel}} = \{S \in \mathcal{F}_3 : |S \cap V_a| = 1 \forall a\}$ as the direct relation-heterogeneity control. We sample one trajectory per validation problem by a fixed hash before any execution, preventing high-volume assignments from dominating selection. In addition to unrestricted f , let R_v^B contain repairs whose AST TED is at most B , and predeclare $\mathcal{B} = \{5, 10, 20, 40, 80, 160\}$. Define $f_B(S) = |\bigcup_{v \in S} R_v^B|/|U|$. The budget-aware objective $f_{\mathcal{B}}(S) = |\mathcal{B}|^{-1} \sum_{B \in \mathcal{B}} f_B(S)$ averages the complete coverage curve for deployments that require one budget-agnostic triple. When the budget is declared before a query, the aligned controller instead freezes the validation maximizer for that budget and invokes its triple under the corresponding gate. We exactly enumerate all $\binom{9}{3} = 84$ members of $\mathcal{F}_3$ and the 27 relation-constrained triples for unrestricted, budget-agnostic, and each budget-indexed objective. The three Answer checkpoints form the prespecified same-target control. Test outcomes never enter selection.

PROPOSITION 2 (COVERAGE STRUCTURE AND EXACT SELECTION). *The objectives f , every f_B , and $f_{\mathcal{B}}$ are normalized, monotone, and submodular. Exhaustive enumeration returns their global optima over $\mathcal{F}_3$, and the parallel enumeration of $V_P \times V_S \times V_A$ returns the global optima under the relation constraint.*

PROOF. Normalization and monotonicity follow from set union. For $S \subseteq T$ and checkpoint $v \notin T$, the marginal gain is $|R_v \setminus \bigcup_{u \in S} R_u|/|U|$, which cannot be smaller than the same quantity with T replacing S ; hence f is submodular. Enumeration evaluates every feasible triple. Each budgeted coverage term has the same set union form, and a nonnegative sum of submodular functions is submodular. The two finite enumerations inspect every member of their respective feasible sets. $\square$

Exact optimization does not by itself guarantee that the validation winner is the best population portfolio. The finite search space nevertheless permits a problem-level guarantee. Let $Z_1, \dots, Z_m$ be independent validation problems drawn from the deployment distribution, let $h_S(Z) \in \{0, 1\}$ indicate whether portfolio S repairs the sampled state for problem Z , and let $F(S) = \mathbb{E}[h_S(Z)]$.

THEOREM 1 (FINITE-PORTFOLIO SELECTION GUARANTEE). *For a frozen family $\mathcal{F}$ of M candidate portfolios, let $\widehat{S}$ maximize empirical problem-level repair coverage on m validation problems and let S^* maximize F . With probability at least $1 - \delta$,*

$$F(S^*) - F(\widehat{S}) \leq \sqrt{\frac{2 \log(2M/\delta)}{m}}. \quad (6)$$

The same statement holds for a fixed edit budget after replacing h_S by the budget-gated repair indicator; simultaneous selection over q declared budgets replaces M by Mq .

PROOF. For any fixed S , Hoeffding's inequality bounds the deviation between its empirical and population coverage by $2e^{-2m\epsilon^2}$. A union bound over the M portfolios makes all deviations at most $\epsilon = \sqrt{\log(2M/\delta)/(2m)}$ simultaneously. On that event, $F(S^*) \leq \widehat{F}(S^*) + \epsilon \leq \widehat{F}(\widehat{S}) + \epsilon \leq F(\widehat{S}) + 2\epsilon$, which gives Equation 6. The budgeted cases use the same binary indicator and a union bound over the declared objectives. $\square$

The theorem makes the unit of selection explicit: sampling one state per validation problem supports a problem-distribution claim, whereas treating all trajectories as independent would not. Its i.i.d. premise is an idealization; we therefore freeze the selected portfolios and report independent test results with problem-clustered intervals rather than using the bound as an effect estimate.

4.7 Execution-Guided Sequential Repair

At inference, we invoke the three validation-selected checkpoints in fine-to-broad relation order, breaking same-relation ties by their frozen member order. Each checkpoint receives the same observed student trajectory and produces one complete program; generated candidates are never inserted into another checkpoint's prompt. This independence makes the outputs a policy portfolio rather than a synthetic trajectory whose later states may lie outside the training distribution. In unrestricted mode, an accepted candidate stops the sequence; otherwise executed candidates are compared by the certified selector below. The budgeted mode additionally continues past accepted candidates that exceed the declared structural limit. The order is fixed before evaluation and tested independently in RQ3–RQ6. We also evaluate the previously plausible alternative of appending generated execution feedback as a controlled ablation.

If all candidates fail, the selector includes the current program c_i as a fallback:

$$C = \{c_i, c^{(1)}, c^{(2)}, c^{(3)}\}. \quad (7)$$

Let $\text{Pass}(c)$ be the fraction of tests passed and $\text{ted}(c_i, c)$ the Python AST tree edit distance. Selection is lexicographic:

$$c^* = \arg \max_{c \in C} (\text{Pass}(c), -\text{ted}(c_i, c)). \quad (8)$$

The selector first maximizes executed correctness and then minimizes change. If no candidate improves on c_i , the unchanged fallback has TED zero and the system abstains. Sequentially generating, executing, and stopping at the first accepted candidate preserves conditional independence given the authentic observed trajectory; if all fail, every executed candidate remains available to Equation 8.

Certified selection and cost. Including c_i in Equation 8 makes observed-test non-regression an invariant: an early return has pass rate one, and otherwise the maximum cannot score below its fallback. With a structural budget B , the gate G_B discards undefined or over-budget TED, continues past an over-budget acceptance, and selects from the eligible candidates plus c_i . Thus every returned change has pass rate at least $\text{Pass}(c_i)$ and $\text{ted}(c_i, c^*) \leq B$; deployed coverage is exactly the validation

set-union objective f_B . These guarantees concern observed tests and edit compliance, not hidden-test correctness or learned relational behavior.

With cached outcomes, constructing a length- n trajectory over T tests costs $O(nT)$ time and $O(n)$ retained-state storage; Answer construction is $O(n)$. Inference performs one to three generations and executions. Early acceptance can reduce this cost, while the cache avoids repeated execution of identical candidates without changing the worst case.

5 Experimental Design

5.1 Research Questions

RQ1: Repair Effectiveness. How effectively does ZPDPATCH repair held-out trajectories from problems observed during training, compared with zero-shot and retrieval-based repair?

RQ2: Trajectory Conditioning. Does access to the full submission prefix improve repair over training and inference with only the current code?

RQ3: Adapter Specialization. Do PROGRESS, STRICT, and ANSWER exhibit different repair coverage and patch size?

RQ4: Sequential Escalation. Does execution-guided composition improve effectiveness without always invoking all adapters?

RQ5: Problem Generalization. Does ZPDPATCH retain an advantage on problems completely excluded from training?

RQ6: Robustness and Transfer. Are the conclusions robust to training seed, user overlap, problem-level dependence, policy ordering, and a student-held-out Java course dataset?

5.2 Dataset Construction and Splits

We combine Python submissions, statements, metadata, and test cases for problems in Project CodeNet’s Python800 benchmark [Puri et al. 2021]. We retain trajectories with at least three submissions, at least one non-accepted state before the first acceptance, and a final accepted program present in the Python800 reference set. We remove post-acceptance submissions and consecutive normalized duplicates. Users must appear in at least three problems and each retained problem must contain at least two trajectories. The refined corpus contains 546 problems, 21,454 trajectories, 99,432 submissions, and 10,088 tests.

Before splitting, we tokenize every possible ANSWER, STRICT, conservative PROGRESS, and final-evaluation prompt-target configuration. If any configuration exceeds 4,096 tokens, we exclude the entire trajectory rather than truncating code or history. This removes 2,896 trajectories and leaves 18,558. No later `max_history`, `transition-count`, `code`, or `test` truncation is applied.

We sort problems by eligible trajectory count and assign the top 90% (492 problems) to *Seen* and the remaining 54 low-resource problems to *Unseen*. Within every *Seen* problem, we reserve at least one trajectory for Train, Validation, and Test, then obtain a deterministic 80:10:10 trajectory split. All 260 *Unseen* trajectories remain problem-held-out test data. Table 2 reports the resulting corpus. Users may occur in more than one split; RQ5 is problem-held-out, not user-held-out.

Adapter construction yields the training and validation sizes in Table 3. Final evaluation uses the prefix ending at the last non-accepted submission and the final accepted program as the oracle. We retain only cases for which re-execution confirms the current program as non-accepted and the oracle as accepted, leaving 997 *Seen* instances from 328 of the 492 *Seen* problems and 250 *Unseen* instances from all 54 held-out problems. The remaining *Seen* problems still contribute Train or Validation trajectories but have no Test trajectory satisfying both execution checks; this is why the abstract’s test-problem count is smaller than the *Seen* pool.

Table 2. Dataset statistics after whole-trajectory context filtering. “Prefix” counts possible raw prefixes before adapter-specific filtering.

Group	Role	Problems	Tests	Traj.	Sub.	Prefix	Users
Seen	Train			14,638	58,872	44,234	4,072
	Validation	492	9,446	1,830	7,283	5,453	1,283
	Test			1,830	7,068	5,238	1,291
Unseen	Test	54	642	260	965	705	237

Table 3. Adapter supervision datasets. Validation examples are excluded from training.

Adapter	Train examples	Validation examples
PROGRESS	21,416	2,685
STRICT	16,973	2,116
ANSWER	40,454	4,965

To test the educational and language scope directly, we add TIKTOC’s testcase-augmented CodeWorkout release, which contains authentic CS1 Java submissions and per-test outcomes [Duan et al. 2025]. We apply the same first-acceptance, duplicate, minimum-length, and 4,096-token whole-trajectory rules, then split by student rather than trajectory. The resulting external corpus has 1,125 trajectories from 224 students over all 17 exercises, with zero student overlap: 786/158/181 train/validation/test trajectories from 157/34/33 students. Every split contains all 17 exercises. External training contains 3,470 Answer, 1,110 Strict, and 1,386 Progress examples; validation contains 730/230/276. We train the same model and hyperparameters from the same base checkpoint. For each relation, seeds 2027/2028/2029 create nine independent checkpoints. We execute all nine on the 158 student-disjoint validation states, exactly enumerate all 84 size-three and 27 one-per-relation portfolios under Equation 5, and freeze both winners before testing. The 181 student-held-out final states compare both with the mandatory same-target Answer-3Seed control; no CodeWorkout test outcome enters checkpoint or portfolio selection.

5.3 Baselines

Zero-shot. The zero-shot baseline uses the same Qwen base model and prompt as ZPDPATCH without fine-tuning. If a candidate fails, its verdict and number of passed tests are added to the conversation before the next attempt. It receives at most three generations, matching ZPDPATCH’s maximum budget. Because it receives the full trajectory, comparison with zero-shot measures the complete effect of learned adapters and sequential composition, not history in isolation.

LSGen. LSGen is an iterative retrieval-based solution generator [Dai et al. 2026]. We preserve its repair-pair filtering, edit-vector retrieval, diff-guided generation, and re-retrieval, while replacing hard-coded infrastructure with our common dataset and runner. For a Seen query, the retrieval database contains buggy/correct pairs from other Seen-Train trajectories of the same problem; the same student and example are excluded. LSGen retrieves five pairs and performs at most three iterations. We do not evaluate it on Unseen problems because exposing same-problem correct programs would violate problem holdout.

All approaches use the same base LLM, public tests, 2.5-second per-test timeout, 2GB memory limit, and a maximum of three candidate generations. Final validation-selected, external-transfer,

and budget-controller runs cap every new candidate at 4,096 new tokens; reused control candidates are audited against the same bound.

5.4 Implementation

We use Qwen2.5-Coder-7B-Instruct [Hui et al. 2024] and train each adapter for one epoch with 4-bit QLoRA [Dettmers et al. 2023]. The base weights use NF4 double quantization and BF16 computation. LoRA targets the q, k, v, o attention projections and gate/up/down MLP projections with rank 16, scaling 32, and dropout 0.05. Optimization uses paged 8-bit AdamW, learning rate 2×10^{-4} , cosine scheduling, 0.03 warmup ratio, and gradient checkpointing. Both canonical and CodeWorkout training use micro-batches of 2 with eight accumulation steps, for an effective batch of 16. Seed 2027 is primary; seeds 2028 and 2029 repeat all adapter trainings and evaluation with identical within-dataset data, hyperparameters, and greedy decoding. We select the lowest validation-loss checkpoint and run on one NVIDIA RTX 5090.

OpenAI Codex was used interactively to propose control and validation-selection design alternatives, implement dataset builders, orchestrate experiments, develop statistical-analysis scripts, and assist manuscript drafting. The authors selected the final design and acceptance criteria, executed every job on author-controlled infrastructure, reviewed code changes, and checked reported values against saved machine-readable outputs. AI-produced text was not treated as empirical evidence, and every cited work was checked against a retrievable source.

An isolated runner executes original and generated programs: Python800 uses the Python judge and CodeWorkout compiles Java 21 in a network-disabled container. We cache outcomes by problem, code, test set, timeout, and memory configuration and reuse the cache across supervision construction and evaluation. Within an evaluation split, every policy is bound to the same cached current-program verdict and pass rate; only a previously unseen generated candidate is executed, which prevents timeout-boundary noise from changing the baseline across portfolio members. The Java harness reproduces all 181 recorded accepted test programs before model evaluation.

5.5 Ablations

For RQ2, we construct adapter-specific STRICT and PROGRESS examples for both Seen and Unseen tests. *Full Trajectory* uses each retained prefix. *Current Code Only* retrains an adapter on identical examples and targets after removing every submission before the current one; its displayed position is reset to S_1 to prevent index leakage. ANSWER is excluded because its inputs already contain one submission. This design isolates the presence of history while holding targets, counts, model, and hyperparameters fixed.

For RQ3, each adapter independently generates one candidate for the same 997 Seen instances. For RQ4, we first compose the seed-2027 candidates with PROGRESS-STRICT-ANSWER early stopping and fallback selection, then evaluate the final validation-selected size-three portfolio. Four controls separate target heterogeneity from repeated calls, prompt-state effects, training randomness, and the relation constraint. *Answer×3* invokes the same Answer adapter up to three times, appending generated execution feedback so that deterministic greedy decoding can produce a non-identical later candidate; repeating the identical prompt would collapse exactly to Answer once. *Generated Feedback* applies the same update protocol to the heterogeneous P-S-A policies. *Answer-3Seed* independently trains three Answer adapters on the identical 40,454 training and 4,965 validation examples with identical targets and hyperparameters, changing only seeds 2027/2028/2029; all three receive the same authentic evaluation trajectory without generated feedback. *Validation Relation-Constrained* selects one checkpoint per relation, while the primary *Validation Execution Portfolio* selects any three of all nine checkpoints on the same problem-balanced validation execution; both are frozen before test. Every independent policy remains conditioned on the same authentic

trajectory. All configurations share the maximum three-generation budget, execution limits, early stopping, and fallback selector. We audit matching Answer training metadata and pairwise-distinct adapter-weight hashes. Effectiveness-only controls may omit AST TED only when failed pass-rate ties are resolved by earliest candidate, which cannot change PR, RR, or IR; Answer-3Seed and the validation-selected portfolios instead use the complete production selector so patch size can also be compared.

5.6 Metrics and Statistical Analysis

For M instances, let c_m be the current program and $\hat{c}_m$ the selected result. $\text{Pass}(c) \in [0, 1]$ is the test pass fraction. Pass Rate (PR) averages $\text{Pass}(\hat{c}_m)$; Repair Rate (RR) averages $1[\text{Pass}(\hat{c}_m) = 1]$; Improvement Rate (IR) averages $1[\text{Pass}(\hat{c}_m) > \text{Pass}(c_m)]$.

For successfully repaired, parseable programs, $\text{TED}_{B \rightarrow F}$ measures AST distance from current to fixed code and $\text{TED}_{F \rightarrow O}$ from fixed code to the recorded accepted oracle, using the standard unit-cost ordered-tree edit formulation [Zhang and Shasha 1989]. Legacy fixed-policy reports use ZSS; validation-selected portfolios and exhaustive budget replay use the equivalent APTED implementation [Pawlik and Augsten 2016], cross-checked against ZSS on identical labeled ASTs. Because node-count difference lower-bounds unit-cost TED, pairs whose difference already exceeds the largest declared budget are stored only as > 160 , which preserves every budget decision. RQ2 uses $\text{TED}_{F \rightarrow T}$, where T is the identical adapter-specific recorded target shared by Full and Current configurations.

We report Wilson 95% intervals for RR and IR [Wilson 1927]. Paired RR comparisons use two-sided exact McNemar tests [McNemar 1947], with Holm correction within each planned comparison family [Holm 1979]. Because trajectories from one problem are dependent, our primary robustness analysis resamples problems as clusters 10,000 times with seed 2027 and reports intervals for paired RR, PR, and IR differences. We additionally report a problem-balanced estimand that gives every problem equal weight, with a problem bootstrap interval. Paired TED differences use 10,000 instance bootstrap resamples on jointly repaired inputs; both procedures follow the nonparametric bootstrap principle [Efron 1979]. Unless stated otherwise, percentages are absolute percentage points.

6 Results

6.1 RQ1: Repair Effectiveness

Table 5 summarizes RQ1 and RQ5. On Seen problems, the validation-selected execution portfolio repairs 613 of 997 programs (61.5%), compared with 306 (30.7%) for Zero-shot. Their Wilson RR intervals are [58.4, 64.5] and [27.9, 33.6], respectively. ZPDPATCH alone repairs 342 instances, whereas Zero-shot alone repairs 35 (exact McNemar $p < 10^{-63}$). More importantly for dependent trajectories, the 30.8-point paired RR gain remains under problem-cluster bootstrap [26.4, 35.1]; the equal-problem-weight gain is 26.1 points [21.2, 30.9]. PR increases by 11.07 points [9.16, 13.02] and IR by 25.2 points [20.8, 29.4] under the same clustered analysis.

The legacy LSGen controller achieves the highest Seen execution correctness: 88.0% PR, 80.2% RR, and 82.4% IR. It alone repairs 267 instances, while ZPDPATCH alone repairs 80 (exact McNemar $p < 10^{-23}$); the clustered RR difference is -18.8 points $[-23.0, -14.5]$. The tradeoff is patch size. ZPDPATCH's mean $\text{TED}_{B \rightarrow F}$ is 21.05, versus 27.25 for Zero-shot and 88.27 for LSGen. Among jointly repaired parseable instances, ZPDPATCH reduces TED by 8.46 relative to Zero-shot (95% CI [5.05, 12.55]) and by 43.34 relative to LSGen (95% CI [38.47, 48.44]). RQ6 replaces the legacy early-stop replay with an always-three LSGen controller before making the final budget-frontier comparison.

Table 4 operationalizes this tradeoff rather than treating TED as evidence of pedagogical quality. Under a maximum structural edit budget, ZPDPATCH has substantially higher repair coverage through $B = 80$; LSGen crosses over only at the broadest reported budget. At $B = 20$, for example, the gate returns an accepted repair for 44.0% of inputs with ZPDPATCH versus 13.9% with always-three LSGen, a +30.09-point problem-clustered difference [26.45, 33.82]. Across all six predeclared budgets, the mean difference is +14.68 points [12.03, 17.43]. The comparison therefore supports a constrained deployment claim: retrieval has higher unrestricted coverage, whereas the trajectory-trained portfolio covers more inputs when large structural replacement is disallowed.

Paired outcomes show complementary success rather than marginal exchanges. On Seen, ZPDPATCH and Zero-shot repair 271 jointly, with 342 versus 35 unique repairs; against LSGen the corresponding counts are 533 jointly and 80 versus 267 uniquely. On Unseen, ZPDPATCH and Zero-shot repair 142 jointly, with 45 versus 13 unique repairs. Thus LSGen has higher coverage, but neither Seen success set contains the other.

PR and IR confirm that the RR gain is accompanied by partial behavioral improvement. Patch distance separates repair from replacement: mean current-to-fix TED is 22.8% below Zero-shot and 76.2% below legacy LSGen on each method’s successful set, while the jointly repaired paired analyses above establish the same direction without success-set confounding.

6.2 RQ2: Trajectory Conditioning

Table 6 shows no consistent advantage for Full Trajectory. On Seen STRICT examples, Full raises PR by 0.7 points but lowers RR and IR by 0.7 and 1.3. On Seen PROGRESS, it lowers PR, RR, and IR by 1.0, 0.4, and 0.6. On Unseen data, Full is worse for STRICT but better for PROGRESS. Target TED is also mixed: Full is closer to the recorded target for Seen PROGRESS and Unseen STRICT, but farther in the other conditions. None of the four paired RR differences is significant after Holm correction (minimum adjusted $p = 0.313$).

Each Full–Current pair holds training examples, targets, architecture, hyperparameters, and evaluation inputs fixed; only earlier submissions are removed and position leakage reset. Direction changes across conditions, clustered RR intervals include zero (Unseen Strict reaches zero at its upper endpoint), and target TED is mixed. RQ1 can therefore establish complete-system utility while RQ2 rejects history serialization as a stable causal explanation.

6.3 RQ3: Adapter Specialization

Table 7 exposes a coverage-size tradeoff. ANSWER has the highest PR, RR, and IR, but also the largest mean patch at TED 20.22. PROGRESS has lower coverage but the smallest patch at TED 15.85. STRICT and PROGRESS have statistically indistinguishable RR (adjusted $p = 0.857$). ANSWER repairs 5.2 points more than PROGRESS and 5.5 points more than STRICT; problem-cluster intervals are [2.5, 7.9] and [2.9, 8.0], respectively. The ordered increase in patch size is consistent with broader targets, but STRICT does not form a clearly distinct coverage regime.

Coverage specialization is weak between PROGRESS and STRICT, but patch-scope specialization is ordered: their mean current-to-fix TEDs are 15.85, 17.45, and 20.22 through Answer, while Answer repairs 5.2–5.5 points more. Oracle distance follows the same pattern. These are distributional effects rather than decoding constraints; hard non-regression enters only through selection.

6.4 RQ4: Sequential Escalation

Independent-policy escalation reaches 59.5% RR, 9.4 points above the strongest individual adapter, ANSWER; its problem-cluster interval [7.6, 11.3] excludes zero. Gains over PROGRESS and STRICT are 14.6 [12.3, 17.0] and 14.9 [12.8, 17.1] points, respectively. The increase appears in PR and IR as well, so the portfolio is not trading complete repairs for weaker partial outputs.

Table 4. Seen repair rate (%) after the same AST-TED deployment gate. ZPDPATCH uses the validation-frozen budget-indexed unconstrained portfolio; LSGen always generates three candidates. The last column is ZPDPATCH minus LSGen with a 95% problem-cluster interval.

Budget B	ZPDPATCH	LSGen	Difference [95% CI]
5	22.5	4.8	+17.7 [14.8, 20.6]
10	32.4	8.0	+24.4 [21.2, 27.7]
20	44.0	13.9	+30.1 [26.4, 33.8]
40	50.9	30.4	+20.5 [17.0, 24.0]
80	56.0	50.8	+5.2 [1.3, 9.3]
160	57.9	67.6	-9.7 [-14.2, -5.4]

Table 5. Main repair results. PR, RR, and IR are percentages; TED is computed on successfully repaired, parseable programs.

Split	Method	PR	RR	IR	$TED_{B \rightarrow F}$	$TED_{F \rightarrow O}$
Seen	Zero-shot	76.3	30.7	43.7	27.25	27.67
	LSGen	88.0	80.2	82.4	88.27	83.23
	ZPDPATCH	87.4	61.5	68.9	21.05	21.64
Unseen	Zero-shot	75.3	62.0	68.8	17.46	15.80
	ZPDPATCH	84.9	74.8	79.6	11.40	11.96

Table 6. RQ2 trajectory-conditioning results. PR, RR, and IR are percentages. Current and Full use identical examples and targets.

Split	Adapter	Input	N	PR	RR	IR	$TED_{F \rightarrow T}$
Seen	STRICT	Current	2,151	56.8	31.2	54.6	40.23
	STRICT	Full	2,151	57.5	30.5	53.3	41.05
	PROGRESS	Current	2,674	58.4	26.3	49.7	36.23
	PROGRESS	Full	2,674	57.4	25.9	49.1	33.15
Unseen	STRICT	Current	322	66.7	56.2	71.4	20.20
	STRICT	Full	322	65.4	53.4	70.8	19.27
	PROGRESS	Current	378	65.7	52.1	66.4	17.63
	PROGRESS	Full	378	66.5	53.7	68.3	18.16

Table 7. RQ3 individual-adapter results on 997 Seen instances.

Adapter	PR	RR	IR	$TED_{B \rightarrow F}$	$TED_{F \rightarrow O}$
PROGRESS	73.4	44.8	52.0	15.85	15.89
STRICT	74.8	44.5	51.2	17.45	18.38
ANSWER	77.5	50.1	57.5	20.22	20.61

Progress accepts 447 inputs, Strict accepts 60 more, and 490 reach Answer. The resulting 2,037 generations average 2.04 per input, avoiding 954 (31.9%) of an always-three budget. Final selection

returns the unchanged program 329 times, making certified abstention a substantial behavior rather than a corner case.

Answer to RQ4. Execution-guided escalation improves both complete repair and partial improvement over every single adapter. Repeating Answer with generated feedback reaches 55.2% RR, so additional calls recover 5.0 points over Answer once but remain 4.31 points below the relation-heterogeneous portfolio. The stricter target-matched control reverses that comparison: three independently trained Answer adapters reach 61.7% RR, exceeding Independent Policies by 2.21 points with a problem-cluster interval [0.11, 4.20] (67 Answer-3Seed-only versus 45 independent-policy-only repairs; McNemar $p = 0.0467$). On jointly repaired programs, Independent Policies makes smaller patches by 1.56 TED [0.36, 2.79], but it does not recover the coverage difference.

Validation execution does not rescue the relation constraint on Seen test. The unrestricted relation-constrained winner obtains 57.3% RR, 4.41 points below Answer-3Seed [-6.54, -2.24]. The unconstrained winner (Answer-2027, Answer-2028, Progress-2027) reaches 61.5%, statistically indistinguishable from Answer-3Seed at -0.20 points [-1.83, 1.46]. This control is decisive: checkpoint complementarity transfers, but forcing one member from every relation reduces coverage. Early stopping still avoids a fixed three-generation cost, and Equation 5 remains the test-independent mechanism for selecting the deployed checkpoints.

Under the six predeclared budgets, choosing a separate validation winner for each declared budget gives the unconstrained family +0.75 mean points over Answer-3Seed [-0.29, 1.86] on Seen. The only individually conclusive budget is TED 20 (+1.81 [0.53, 3.16]); the relation-constrained family averages -0.70 [-2.02, 0.71]. Thus the CodeNet evidence supports budget-indexed validation selection at a specific operating point, not a general coverage benefit from relation heterogeneity.

6.5 RQ5: Problem Generalization

On 250 Unseen instances, ZPDPATCH obtains 84.9% PR, 74.8% RR, and 79.6% IR, versus 75.3%, 62.0%, and 68.8% for Zero-shot. ZPDPATCH alone repairs 45 cases and Zero-shot alone repairs 13 (McNemar $p = 3.01 \times 10^{-5}$). Problem-cluster intervals exclude zero for RR (+12.8 points [6.83, 18.75]), PR (+9.56 [5.66, 13.64]), and IR (+10.8 [4.76, 16.95]); the equal-problem-weight RR gain is 12.38 points [6.01, 18.62]. ZPDPATCH also reduces mean successful-patch TED from 17.46 to 11.40. Among 122 jointly repaired parseable instances, its patch is 6.41 TED units smaller on average (95% CI [3.43, 9.89]).

Against the weaker repeated-Answer-with-feedback control on Unseen data, Independent Policies reaches 73.2% versus 68.4% RR, with 15 portfolio-only and three control-only repairs (McNemar $p = 0.00754$). The independent Answer-3Seed control, however, also reaches 73.2%; its problem-clustered difference from Independent Policies is 0.0 points [-3.2, 3.1] and McNemar $p = 1.0$. The problem-held-out result therefore supports execution-selected checkpoint diversity but does not identify target-relation heterogeneity as its source.

The frozen validation winners reach 74.8% Unseen RR for both unrestricted relation-constrained and unconstrained selection, +1.60 points over Answer-3Seed; both unrestricted intervals include zero. Budget-indexed unconstrained selection is stronger across the declared frontier: its mean coverage difference is +2.60 points [0.38, 5.03], whereas the relation-constrained mean is +1.60 [-1.07, 4.44]. This independently confirms the budget-selection effect but again does not support the one-per-relation constraint.

The 45 favorable versus 13 unfavorable discordant pairs against Zero-shot show complementary successes rather than dominance. Absolute Unseen RR is higher for both methods, but the split was defined by trajectory availability rather than randomized difficulty. The valid estimand is the paired within-Unseen difference, which remains positive without same-problem training.

Table 8. RQ4 sequential and validation-selected portfolios on 997 Seen instances. Selection uses one validation trajectory per problem; test outcomes are not used.

Configuration	PR	RR	IR
Answer×3	84.1	55.2	62.4
Answer-3Seed	87.0	61.7	69.1
Generated Feedback	84.9	56.6	64.6
Fixed P-S-A	86.5	59.5	67.0
Validation relation-constrained	85.8	57.3	65.4
Validation unconstrained	87.4	61.5	68.9

Answer to RQ5. ZPDPATCH’s advantage over Zero-shot persists on problems completely excluded from adapter training. Because the Unseen group deliberately contains lower-resource problems, its higher absolute RR must not be interpreted as evidence that unseen problems are easier or that familiarity harms repair.

6.6 RQ6: Robustness and External Transfer

Implementation conformance. An artifact audit finds zero construction-predicate violations and zero selected-pass-rate regressions; these are specification checks, not learned-behavior evidence.

Because users may cross CodeNet splits, we stratify by training-user overlap. Among 155 Seen instances from 150 training-disjoint users, ZPDPATCH gains 40.0 RR points over Zero-shot [30.7, 48.8]; the 842-instance overlap stratum gains 26.7 [22.6, 31.0]. The 69 Unseen disjoint-user instances retain +11.6 points, but their interval [−1.5, 25.4] includes zero. These observational strata do not identify a memorization effect.

Complete P-S-A retraining with seeds 2027/2028/2029 gives Seen RR 59.5/57.6/59.0% (SD 1.0 points) and Unseen RR 73.2/74.0/70.4% (SD 1.9). Replaying all six policy orders leaves fixed-candidate RR unchanged; P-S-A has the lowest mean breadth (1.391) and successful-patch TED (21.10).

Student-held-out Java transfer. Validation selects A29–P29–S28 under the relation constraint and A28–A29–S28 without it; Answer-3Seed is A27–A28–A29. Table 9 reports the prospective 181-trajectory CodeWorkout test. Unconstrained selection transfers at 84.0% RR but does not beat Answer-3Seed (84.5%): −0.55 points, with exercise/student-cluster CIs [−3.06, 1.82]/[−2.89, 1.93]. The relation constraint reaches 81.8%; no planned contrast is significant after Holm correction (minimum adjusted $p = 0.539$).

Answer to RQ6. Problem-clustered inference preserves the CodeNet effects and three full runs bound training-seed variation. Student-held-out Java evaluation establishes cross-language, course-data transfer of high repair coverage, but it does not establish a coverage advantage over same-target checkpoint diversity.

6.7 Cross-RQ Synthesis

Together, the RQs distinguish properties of the data construction from causes of test-time coverage. The execution-evidence orders are exactly satisfied by the materialized supervision, but neither the history ablation nor the Answer-3Seed control shows that relation heterogeneity causes the portfolio gain. What the experiments do establish is that independently trained checkpoints provide complementary candidates and that execution controls escalation, abstention, and the optional structural budget. The validation-execution selector converts that complementarity into a test-independent deployment choice. Its unconstrained winner matches Answer-3Seed on Seen and

Table 9. Student-held-out CodeWorkout Java transfer (181 trajectories, 33 students, 17 exercises); values are percentages.

Validation-frozen portfolio	PR	RR	IR
Relation-constrained	95.5	81.8	86.2
Unconstrained	96.3	84.0	87.8
Answer-3Seed	96.9	84.5	89.5

improves the mean Unseen budget frontier, while the explicit relation constraint is detrimental on Seen. CodeWorkout reproduces high portfolio coverage under student and language shift, but not an advantage over Answer-3Seed.

7 Discussion

7.1 What Produces the Observed Improvement?

Adapter training matters because every trained portfolio beats a zero-shot model that already receives trajectory context, and sequential execution matters because composition beats every member. The stronger causal attribution is checkpoint diversity: Answer-3Seed exceeds the original relation-heterogeneous portfolio on Seen and ties it on Unseen. Relation-mined targets still change the patch-scope distributions, but the present controls do not establish them as a coverage mechanism. Our final method therefore lets validation execution select any three checkpoints and reports both the one-per-relation restriction and the seed-matched Answer portfolio as mandatory comparators. The selected CodeNet portfolio contains two Answer checkpoints and Progress, empirically rejecting the assumption that all three target relations must appear. Counterfactual histories paired with identical current code would further test when path information changes the appropriate repair rather than only its supervision source.

7.2 Coverage Versus Patch Size

LSGen demonstrates the value of same-problem correct programs, reaching higher unrestricted coverage with mean TED over four times ZPDPATCH’s. Smaller edits do not prove pedagogical superiority. The budget gate instead gives edit size an operational meaning: ZPDPATCH repairs more inputs than LSGen at every tested budget through TED 80, whereas LSGen dominates when broad changes are allowed. This is a deployment coverage–intervention frontier, not a learner-outcome claim.

7.3 Implications for Educational Use

ZPDPATCH produces programs, not hints; deployment should expose diffs or derive a hint ladder rather than overwrite work. Its fallback prevents presenting observed-test regressions as help. A learner study measuring transfer, time, and help seeking is still required to establish educational benefit; the present contribution is a repair and feedback-authoring substrate.

8 Threats to Validity

Construct validity. Passing the available tests is a proxy for semantic correctness and may admit overfitting, a longstanding generate-and-validate repair risk [Qi et al. 2014; Smith et al. 2015]. We reduce this risk by requiring the recorded oracle to pass the same suite, but hidden tests could change absolute rates. AST TED measures structural change, not readability, conceptual difficulty, or educational value. “Reachable” means observed later in a trajectory; it does not imply that the

generated transition lies within an individual learner’s ZPD. We therefore avoid claims about learning outcomes.

Internal validity. Dataset construction, verdict ordering, normalization, runner limits, and candidate selection can affect results. We use one runner and shared outcome cache across methods, preserve full trajectories after filtering, and compare paired outputs on identical instances. LSGen required infrastructure adaptation; although its retrieval and generation logic is retained, our integration may differ from its original environment. Greedy decoding removes sampling variance, and three full QLoRA runs quantify training-seed variation, but other hyperparameters and stochastic decoding remain untested.

Data leakage and dependence. Unseen problems are completely excluded from training and retrieval. This problem-level separation follows evidence that duplicated code across machine-learning splits can materially inflate code-model evaluation [Allamanis 2019]. Seen splits share problem identities by design, and users may occur across splits; the benchmark is not user-held-out. We address assignment-level dependence with problem-cluster bootstrap and an equal-problem-weight estimand. The Seen paired gain persists among 155 training-disjoint-user instances, but the smaller Unseen disjoint-user interval includes zero; a prospectively user-held-out split would more directly test personalization transfer.

External validity. The study covers Python contest submissions from Project CodeNet and student-held-out Java CS1 submissions from CodeWorkout, but only one 7B code model and one GPU. Results may not transfer to multi-file projects, industrial defects, private autograders, other course structures, or other model families. The CodeNet Unseen set consists of lower-volume problems by construction and should not be treated as a random sample of all new assignments.

Conclusion validity. We define explicit metric families, correct planned RR comparisons, and repeat primary effect estimates with problem clusters. TED intervals remain conditional on joint successful repair and should not be generalized to failures. The current model family and testcase suite still delimit the measured effects; additional architectures and hidden tests would test whether the ordering transfers.

Ethical considerations. The study analyzes publicly released code datasets and performs no intervention with learners. We do not attempt to re-identify users, and derived evaluation reports need only pseudonymous identifiers. Nevertheless, source-code histories can carry authorship signals; a public artifact should minimize identifiers and follow each source dataset’s distribution terms.

9 Conclusion

We presented ZDPATCH, an execution-guided APR framework whose finite execution-evidence orders mine auditable targets from authentic trajectories. Exact validation selection reaches 61.5% Seen and 74.8% problem-held-out Unseen RR, versus 30.7% and 62.0% for zero-shot. Answer-3Seed reaches 61.7/73.2% and 84.5% versus 84.0% on CodeWorkout Java; the controls identify checkpoint diversity, not compulsory relation heterogeneity, as the coverage mechanism. Retrieval has higher unrestricted Seen coverage, while the edit gate raises mean coverage by 14.68 points over always-three LSGen. Exact finite-family selection and frozen testing separate deployment choice from post-hoc seed choice.

Data availability. An anonymized artifact will provide code, prompts, splits, outputs, and a SHA-256 manifest. Project CodeNet and CodeWorkout are reacquired from their official sources; deterministic regeneration and checksums are included.

References

- Vincent Aleven and Kenneth R Koedinger. 2000. Limitations of student control: Do students know when they need help?. In *International conference on intelligent tutoring systems*. Springer, 292–303.
- Miltiadis Allamanis. 2019. The Adverse Effects of Code Duplication in Machine Learning Models of Code. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. 143–153. doi:10.1145/3359591.3359735
- Dongwook Choi, Jinseok Heo, and Eunseok Lee. 2021. Automated Feedback Generation for Multiple Function Programs. In *2021 28th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 582–583.
- Dongwook Choi and Eunseok Lee. 2025. Automated Feedback Generation for Programming Assignments Through Diversification. In *2025 IEEE/ACM 37th International Conference on Software Engineering Education and Training (CSEE&T)*. IEEE, 230–241.
- Irene-Angelica Chounta, Patricia Albacete, Pamela Jordan, Sandra Katz, and Bruce M McLaren. 2017. The “Grey Area”: A computational approach to model the Zone of Proximal Development. In *European conference on technology enhanced learning*. Springer, 3–16.
- Michael Cole, Vera John-Steiner, Sylvia Scribner, and Ellen Souberman. 1978. Mind in society. *Mind in society the development of higher psychological processes*. Cambridge, MA: Harvard University Press (1978).
- Albert T Corbett and John R Anderson. 1994. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User modeling and user-adapted interaction* 4, 4 (1994), 253–278.
- Zhenlong Dai, Zhuoluo Zhao, Hengning Wang, Xiu Tang, Sai Wu, Chang Yao, Zhipeng Gao, and Jingyuan Chen. 2026. Learner-Tailored Program Repair: A Solution Generator with Iterative Edit-Driven Retrieval Enhancement. *arXiv preprint arXiv:2601.08545* (2026).
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. *Advances in neural information processing systems* 36 (2023), 10088–10115.
- Zhangqi Duan, Nigel Fernandez, Alexander Hicks, and Andrew Lan. 2025. Test Case-Informed Knowledge Tracing for Open-ended Coding Tasks. In *Proceedings of the 15th International Learning Analytics and Knowledge Conference*. 235–250. doi:10.1145/3706468.3706500
- Bradley Efron. 1979. Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics* 7, 1 (1979), 1–26. doi:10.1214/aos/1176344552
- Luca Gazzola, Daniela Micucci, and Leonardo Mariani. 2019. Automatic Software Repair: A Survey. *IEEE Transactions on Software Engineering* 45, 1 (2019), 34–67. doi:10.1109/TSE.2017.2755013
- Aritra Ghosh, Neil Heffernan, and Andrew S Lan. 2020. Context-aware attentive knowledge tracing. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*. 2330–2339.
- Elena L. Glassman, Jeremy Scott, Rishabh Singh, Philip J. Guo, and Robert C. Miller. 2015. OverCode: Visualizing Variation in Student Solutions to Programming Problems at Scale. *ACM Transactions on Computer-Human Interaction* 22, 2, Article 7 (2015), 35 pages. doi:10.1145/2699751
- Sumit Gulwani, Ivan Radiček, and Florian Zuleger. 2018. Automated clustering and program repair for introductory programming assignments. *ACM SIGPLAN Notices* 53, 4 (2018), 465–480.
- Md Mahim Anjum Haque, Wasi Uddin Ahmad, Ismini Lourentzou, and Chris Brown. 2023. Fixeval: Execution-based evaluation of program fixes for programming problems. In *2023 IEEE/ACM International Workshop on Automated Program Repair (APR)*. IEEE, 11–18.
- John Hattie and Helen Timperley. 2007. The Power of Feedback. *Review of Educational Research* 77, 1 (2007), 81–112. doi:10.3102/003465430298487
- Hasnain Heckal and Andrew Lan. 2024. Generating feedback-ladders for logical errors in programming using large language models. *arXiv preprint arXiv:2405.00302* (2024).
- Jinseok Heo, Hohyeon Jeong, Dongwook Choi, and Eunseok Lee. 2023. Referent: Transformer-based feedback generation using assignment information for programming course. In *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*. IEEE, 101–106.
- Sture Holm. 1979. A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6, 2 (1979), 65–70. doi:10.2307/4615733
- Yang Hu, Umair Z Ahmed, Sergey Mechtaev, Ben Leong, and Abhik Roychoudhury. 2020. Re-factoring based program repair applied to programming assignments. In *Proceedings-2019 34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019*. IEEE, 388–398.
- Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, et al. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186* (2024).
- Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of Code Language Models on Automated Program Repair. In *Proceedings of the 45th International Conference on Software Engineering*. 1430–1442. doi:10.1109/ICSE48619.2023.00125

- Harshit Joshi, José Cambronero Sanchez, Sumit Gulwani, Vu Le, Gust Verbruggen, and Ivan Radiček. 2023. Repair is nearly generation: Multilingual program repair with llms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 5131–5140.
- Hieke Keuning, Johan Jeuring, and Bastiaan Heeren. 2018. A systematic literature review of automated feedback generation for programming exercises. *ACM Transactions on Computing Education (TOCE)* 19, 1 (2018), 1–43.
- Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2012. GenProg: A Generic Method for Automatic Software Repair. *IEEE Transactions on Software Engineering* 38, 1 (2012), 54–72. doi:10.1109/TSE.2011.104
- Fang Liu, Zhenwei Liu, Qianhui Zhao, Jing Jiang, Li Zhang, Zian Sun, Ge Li, Zhongqi Li, and Yuchi Ma. 2024. Fastfixer: An efficient and effective approach for repairing programming assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 669–680.
- Thibaud Lutellier, Hung Viet Pham, Lawrence Pang, Yitong Li, Moshi Wei, and Lin Tan. 2020. CoCoNuT: Combining Context-Aware Neural Translation Models Using Ensemble for Program Repair. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 101–114. doi:10.1145/3395363.3397369
- Quinn McNemar. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika* 12, 2 (1947), 153–157. doi:10.1007/BF02295996
- Martin Monperrus. 2018. Automatic Software Repair: A Bibliography. *Comput. Surveys* 51, 1, Article 17 (2018), 24 pages. doi:10.1145/3105906
- Tom Murray and Ivon Arroyo. 2002. Toward measuring and maintaining the zone of proximal development in adaptive instructional systems. In *International conference on intelligent tutoring systems*. Springer, 749–758.
- Susanne Narciss. 2008. Feedback strategies for interactive learning tasks. In *Handbook of research on educational communications and technology*. Routledge, 125–143.
- Pedro Orvalho, Mikoláš Janota, and Vasco M Manquinho. 2025. Counterexample guided program repair using zero-shot learning and maxsat-based fault localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 649–657.
- Mateusz Pawlik and Nikolaus Augsten. 2016. Tree edit distance: Robust and memory-efficient. *Information Systems* 56 (2016), 157–173. doi:10.1016/j.is.2015.08.004
- Tung Phung, José Cambronero, Sumit Gulwani, Tobias Kohn, Rupak Majumdar, Adish Singla, and Gustavo Soares. 2023. Generating high-precision feedback for programming syntax errors using large language models. *arXiv preprint arXiv:2302.04662* (2023).
- Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J Guibas, and Jascha Sohl-Dickstein. 2015a. Deep knowledge tracing. *Advances in neural information processing systems* 28 (2015).
- Chris Piech, Jonathan Huang, Andy Nguyen, Mike Phulsuksombati, Mehran Sahami, and Leonidas Guibas. 2015b. Learning Program Embeddings to Propagate Feedback on Student Code. In *Proceedings of the 32nd International Conference on Machine Learning*. 1093–1102.
- Thomas W Price, Yihuan Dong, and Dragan Lipovac. 2017. iSnap: towards intelligent tutoring in novice programming environments. In *Proceedings of the 2017 ACM SIGCSE Technical Symposium on computer science education*. 483–488.
- Ruchir Puri, David S Kung, Geert Janssen, Wei Zhang, Giacomo Domeniconi, Vladimir Zolotov, Julian Dolby, Jie Chen, Mihir Choudhury, Lindsey Decker, et al. 2021. Project codenet: A large-scale ai for code dataset for learning a diversity of coding tasks. *arXiv preprint arXiv:2105.12655* 1035 (2021).
- Yuhua Qi, Xiaoguang Mao, Yan Lei, Ziyang Dai, and Chengsong Wang. 2014. The Strength of Random Search on Automated Program Repair. In *Proceedings of the 36th International Conference on Software Engineering*. 254–265. doi:10.1145/2568225.2568254
- Kelly Rivers and Kenneth R. Koedinger. 2017. Data-Driven Hint Generation in Vast Solution Spaces: A Self-Improving Python Programming Tutor. *International Journal of Artificial Intelligence in Education* 27, 1 (2017), 37–64. doi:10.1007/s40593-015-0070-z
- Lianne Roest, Hieke Keuning, and Johan Jeuring. 2024. Next-step hint generation for introductory programming using large language models. In *Proceedings of the 26th Australasian Computing Education Conference*. 144–153.
- Nadine Schulze Bernd and Irene-Angelica Chounta. 2024. Beyond the grey area: exploring the effectiveness of scaffolding as a learning measure. In *International Conference on Artificial Intelligence in Education*. Springer, 365–378.
- Valerie J. Shute. 2008. Focus on Formative Feedback. *Review of Educational Research* 78, 1 (2008), 153–189. doi:10.3102/0034654307313795
- Edward K. Smith, Earl T. Barr, Claire Le Goues, and Yuriy Brun. 2015. Is the Cure Worse than the Disease? Overfitting in Automated Program Repair. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering*. 532–543. doi:10.1145/2786805.2786825
- Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2019. An Empirical Study on Learning Bug-Fixing Patches in the Wild via Neural Machine Translation. *ACM Transactions on Software Engineering and Methodology* 28, 4, Article 19 (2019), 29 pages. doi:10.1145/3340544

- Ke Wang, Rishabh Singh, and Zhendong Su. 2018. Search, align, and repair: data-driven feedback generation for introductory programming exercises. In *Proceedings of the 39th ACM SIGPLAN conference on programming language design and implementation*. 481–495.
- Westley Weimer, ThanhVu Nguyen, Claire Le Goues, and Stephanie Forrest. 2009. Automatically Finding Patches Using Genetic Programming. In *Proceedings of the 31st International Conference on Software Engineering*. 364–374. doi:10.1109/ICSE.2009.5070536
- Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *J. Amer. Statist. Assoc.* 22, 158 (1927), 209–212. doi:10.1080/01621459.1927.10502953
- Chunqiu Steven Xia and Lingming Zhang. 2022. Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-Shot Learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 959–971. doi:10.1145/3540250.3549101
- Ruiwei Xiao, Xinying Hou, and John Stamper. 2024. Exploring how multiple levels of GPT-generated programming hints support or disappoint novices. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*. 1–10.
- Boyang Yang, Haoye Tian, Weiguo Pian, Haoran Yu, Haitao Wang, Jacques Klein, Tegawendé F Bissyandé, and Shunfu Jin. 2024. Cref: An llm-based conversational software repair framework for programming tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 882–894.
- Michihiro Yasunaga and Percy Liang. 2021. Break-it-fix-it: Unsupervised learning for program repair. In *International conference on machine learning*. PMLR, 11941–11952.
- He Ye, Matias Martinez, Xiapu Luo, Tao Zhang, and Martin Monperrus. 2022. Selfapr: Self-supervised program repair with test execution diagnostics. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–13.
- Jooyong Yi, Umair Z Ahmed, Amey Karkare, Shin Hwei Tan, and Abhik Roychoudhury. 2017. A feasibility study of using automated program repair for introductory programming assignments. In *Proceedings of the 2017 11th joint meeting on foundations of software engineering*. 740–751.
- Jialu Zhang, José Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2022. Repairing bugs in python assignments using large language models. *arXiv preprint arXiv:2209.14876* (2022).
- Jialu Zhang, José Pablo Cambronero, Sumit Gulwani, Vu Le, Ruzica Piskac, Gustavo Soares, and Gust Verbruggen. 2024. Pydex: Repairing bugs in introductory python assignments using llms. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024), 1100–1124.
- Kaizhong Zhang and Dennis Shasha. 1989. Simple Fast Algorithms for the Editing Distance Between Trees and Related Problems. *SIAM J. Comput.* 18, 6 (1989), 1245–1262. doi:10.1137/0218082
- Qianhui Zhao, Fang Liu, Li Zhang, Yang Liu, Zhen Yan, Zhenghao Chen, Yufei Zhou, Jing Jiang, and Ge Li. 2024. Peer-aided repairer: Empowering large language models to repair advanced student assignments. *arXiv preprint arXiv:2404.01754* (2024). doi:10.48550/arXiv.2404.01754